自动售货机微信小程序技术文档

一、系统概述

本项目是一个基于微信小程序的智能售货机系统，通过连接后端API实现商品展示、购买、支付及管理功能。系统采用前后端分离架构，前端使用微信小程序开发框架，后端采用Python Flask框架提供API服务，数据存储使用MySQL数据库。

二、系统架构

1. 前端架构

- **开发框架**：微信小程序开发框架
- **项目标识**：appid: wxa7da99554722c49f
- **主要页面**：首页、商品详情页、购物车、订单列表、订单详情、个人中心、登录/注册页面等

2. 后端架构

- **服务器环境**：宝塔面板管理的服务器（IP：47.108.141.135）
- **开发语言**：Python
- **Web框架**：Flask（监听5001端口）
- **API路径**：`http://47.108.141.135:5001/`
- **服务部署位置**：`/www/wwwroot/file-upload-server/app.py`

3. 数据存储

- **数据库**：MySQL（数据库名：vending_mysql）
- **主要数据表**：
  - `users`：用户信息表
  - `products`：商品信息表

- `orders`：订单表
- `payments`：支付信息表
- `vending_machines`：售货机信息表
- `vending_machine_products`：售货机商品关联表

三、功能模块详解

1. 用户管理模块

1.1 登录功能

- 支持用户名密码登录
- 用户输入验证（非空验证）
- 登录状态保持
- API接口：`/api/user/login`（POST）

1.2 注册功能

- 用户名与密码设置
- 密码强度检测（长度、大小写、数字、特殊字符）
- 密码二次确认
- API接口：`/api/user/register`（POST）

1.3 个人信息管理

- 查看个人信息
- 修改昵称功能
- API接口：`/api/user/info`（GET）、`/api/user/update`（POST）

2. 商品展示模块

2.1 商品列表

- 首页商品网格展示

- 支持下拉刷新获取最新库存
- 轮播图展示
- API接口：`/api/products/list`（GET）

2.2 商品详情

- 商品详细信息展示（名称、价格、库存、描述等）
- 数量选择功能
- 加入购物车和立即购买功能
- 库存状态提示（低库存警告）

3. 购物车模块

3.1 购物车管理

- 添加商品到购物车
- 购物车商品列表展示
- 商品数量调整
- 商品删除功能
- 商品选择/全选功能

3.2 结算功能

- 计算选中商品总价
- 跳转到结算页面
- 生成订单

4. 订单管理模块

4.1 订单列表

- 不同状态订单分类展示（全部、待付款、已完成、售后）
- 订单基本信息展示
- 订单状态标识
- API接口：`/api/orders`（GET）

4.2 订单详情

- 商品信息展示
- 订单状态展示
- 订单时间信息
- 金额信息展示
- API接口：`/api/order/detail`（GET）

4.3 订单操作

- 待付款订单支付功能
- 订单取消功能
- 申请退款功能
- 再次购买功能

5. 支付模块

- 支持多种支付方式
- 支付流程处理
- 支付状态更新
- API接口：`/api/payment/create`（POST）

6. 管理员功能

- 销售数据统计
- 设备状态监控
- API接口：`/data`（GET）

7. 地理位置服务

- 售货机位置展示
- 附近售货机查找
- 地图标记展示

四、API接口设计

1. 用户相关API

接口名称	请求路径	方法	功能描述
用户登录	`/api/user/login`	POST	验证用户名和密码，返回用户信息
获取用户信息	`/api/user/info`	GET	根据用户ID获取用户信息
更新用户信息	`/api/user/update`	POST	更新用户昵称
用户注册	`/api/user/register`	POST	注册新用户

2. 商品相关API

接口名称	请求路径	方法	功能描述
获取商品列表	`/api/products/list`	GET	获取所有商品的详细信息

3. 订单相关API

接口名称	请求路径	方法	功能描述
创建订单	`/api/order/create`	POST	用户下单，检查库存并创建订单
获取用户订单列表	`/api/orders`	GET	根据用户ID获取该用户的所有订单
获取订单详情	`/api/order/detail`	GET	根据订单ID获取订单详细信息

4. 支付相关API

接口名称	请求路径	方法	功能描述
创建支付记录	`/api/payment/create`	POST	用户完成支付，创建支付记录并更新用户余额

5. 设备相关API

接口名称	请求路径	方法	功能描述
获取设备商品列表	`/devices/<device_id>/products`	GET	获取特定设备的商品列表
更新商品库存	`/devices/<device_id>/stock`	POST	更新特定设备的商品库存
货道状态监控	`/devices/<device_id>/slots`	GET	获取特定设备的货道状态
获取设备状态	`/device_status/<device_id>`	GET	获取单个设备的状态信息
批量查询设备状态	`/devices/status`	POST	批量获取多个设备的状态信息

五、数据库设计

1. users表

- `id` : int(11) - 用户ID（主键，自增）
- `username` : varchar(50) - 用户名
- `password` : varchar(225) - 密码
- `balance` : decimal(10,2) - 用户余额

2. products表

- `id` : int(11) - 商品ID（主键，自增）
- `name` : varchar(100) - 商品名称
- `price` : decimal(5,2) - 商品价格
- `stock` : int(11) - 商品库存
- `location` : varchar(20) - 商品货道位置

3. orders表

- `id` : int(11) - 订单ID（主键，自增）
- `user_id` : int(11) - 用户ID（外键关联users表）
- `product_id` : int(11) - 商品ID（外键关联products表）
- `quantity` : int(11) - 购买数量
- `total_price` : decimal(10,2) - 总价
- `created_at` : datetime - 创建时间
- `vending_machine_id` : int(11) - 售货机ID（外键关联vending_machines表）

4. payments表

- `id` : int(11) - 支付ID（主键，自增）
- `users_id` : int(11) - 用户ID（外键关联users表）
- `order_id` : int(11) - 订单ID（外键关联orders表）
- `method` : varchar(50) - 支付方式
- `amount` : int(11) - 支付金额
- `paid_at` : datetime - 支付时间

5. vending_machines表

- `id` : int(11) - 设备ID（主键，自增）
- `status` : varchar(20) - 设备状态
- `name` : varchar(50) - 设备名称

- `locations`: varchar(100) - 设备位置

6. vending_machine_products表

- `vending_machine_id`: int(11) - 售货机ID（联合主键，外键关联vending_machines表）
- `product_id`: int(11) - 商品ID（联合主键，外键关联products表）
- `stock`: int(11) - 特定售货机中的商品库存

六、前端页面组织

1. 页面结构

- **首页** (`pages/index/index`): 商品展示、轮播图、库存信息
- **商品详情页** (`pages/product/detail`): 商品详情、数量选择、购买操作
- **购物车页面** (`pages/cart/index`): 购物车商品管理、结算
- **订单列表页** (`pages/order/list`): 不同状态订单展示
- **订单详情页** (`pages/order/detail`): 订单详细信息、支付操作
- **个人中心页** (`pages/profile/index`): 用户信息、订单统计、功能入口
- **登录页面** (`pages/login/index`): 用户登录
- **注册页面** (`pages/register/index`): 用户注册
- **位置服务页** (`pages/location/index`): 售货机位置展示
- **管理员页面** (`pages/admin/index`): 数据统计和管理
- **用户政策页** (`pages/policy/index`): 使用条款、隐私政策

2. 样式设计

- 整体使用简洁、现代的UI设计风格
- 主色调采用红色系（`#e74c3c`）
- 响应式设计，适配不同屏幕尺寸
- 交互友好，提供适当的加载状态和反馈

七、安全性设计

1. **密码安全**
    - 密码强度校验（长度、复杂度要求）
    - 密码二次确认机制
    - 服务端密码加密存储

2. **数据传输安全**
    - 敏感数据加密传输
    - HTTPS协议保障传输安全

3. **访问控制**
    - 用户身份验证
    - 权限管理

4. **异常处理**
    - 完善的错误捕获和处理机制
    - 友好的错误提示

八、用户隐私保护

遵循《用户政策》中规定的隐私保护措施：

- 明确告知用户收集的信息类型
- 说明信息使用目的
- 采取措施保障信息安全
- 规范信息共享行为
- 尊重用户对个人信息的控制权

九、优化与改进方向

1. **性能优化**
    - 进一步优化加载速度
    - 实现更高效的数据缓存策略

2. **功能扩展**
    - 增加用户评价系统
    - 引入推荐算法，提供个性化商品推荐
    - 增强售货机远程监控能力

3. **用户体验提升**
    - 增加更多支付方式
    - 优化商品搜索和筛选功能
    - 完善订单跟踪系统

十、开发与测试工具

1. **开发工具**
    - 微信开发者工具
    - Visual Studio Code

2. **测试工具**
    - ApiPost（API测试）
    - 微信开发者工具内置调试功能

3. **版本管理**
    - Git版本控制

十一、总结

本智能售货机小程序系统实现了从商品展示、用户交互到订单支付的完整流程，采用前后端分离架构，保证了系统的可扩展性和维护性。系统通过Flask后端与MySQL数据库的连接，实现了数据的实时获取与存储，为用户提供了流畅、便捷的购物体验。未来可以在用户体验、性能优化和功能扩展等方面继续改进，以满足不断变化的市场需求。